returned values to the `IConfigurationRoot`, which implements `IConfiguration`.

NOTE Adding a provider to the `ConfigurationManager` adds the configuration values to the `IConfigurationRoot` instance, which implements `IConfiguration`. You'll generally work with the `IConfiguration` interface in your code.

ASP.NET Core ships with configuration providers for loading data from common locations:

- JSON files
- Extensible Markup Language (XML) files
- Environment variables
- Command-line arguments
- Initialization (INI) files

If these providers don't fit your requirements, you can find a host of alternatives on GitHub and NuGet, and it's not difficult to create your own custom provider. You could use the official Microsoft Azure Key Vault provider NuGet package or the YAML file provider I wrote.

NOTE The Azure Key Vault provider is available on NuGet at http://mng.bz/0KrN, and you can find my YAML provider on GitHub at http://mng.bz/Yqdj.

In many cases, the default providers are sufficient. In particular, most templates start with an appsettings.json file, which contains a variety of settings depending on the template you choose. The following listing shows the default file generated by the ASP.NET Core 7.0 Empty template without authentication.

Listing 10.1 Default appsettings.json file created by an ASP.NET Core Empty template